MODUL 18 · BAB VI · ALGORITMA GRAF

Depth-First Search (DFS)

Menelusuri graf sedalam mungkin lalu mundur (backtrack), mencatat waktu ditemukan dan waktu selesai setiap simpul.

TUJUAN PEMBELAJARAN

Setelah mempelajari modul ini, kamu diharapkan mampu:

- Menjelaskan cara kerja DFS: penelusuran mendalam dan backtrack.
- Menjelaskan arti timestamp $u.d$ (discovery time) dan $u.f$ (finishing time).
- Menelusuri prosedur DFS dan DFS-VISIT pada graf berarah secara lengkap.
- Mengklasifikasikan busur menjadi tree, back, forward, dan cross, serta menggambar DFS-forest.
- Menjelaskan running time $\Theta(V + E)$ dan perbedaan DFS dengan BFS.

PRASYARAT

- Modul 16: graf dan adjacency-list.
- Modul 17: BFS (sebagai pembanding).
- Modul 02: rekursi dan call stack.

BAB / MATERI ASAL

Materi 11.2 ALGORITMA GRAF, DFS

POSISI DALAM URUTAN BELAJAR

Algoritma penelusuran graf kedua. Menutup bagian penelusuran sebelum masuk ke Minimum Spanning Tree (Modul 19).

DAFTAR ISI

1. Menyusuri Labirin 1
2. Cara Kerja DFS 2
3. Klasifikasi Busur 3
4. Penelusuran DFS pada Graf Berarah 3
5. Running Time dan Perbandingan dengan BFS 7
6. Kesimpulan 8
- Kuis Pemahaman 9
- Kunci Jawaban dan Pembahasan 10
- Referensi 10

PENDAHULUAN

1. Menyusuri Labirin

Bayangkan menjelajahi sebuah labirin. Strategi yang wajar: pilih satu lorong dan terus maju sejauh mungkin. Bila menemui jalan buntu atau tempat yang sudah pernah didatangi, **mundur** ke persimpangan terakhir yang masih punya lorong yang belum dicoba, lalu maju lagi dari sana. Strategi ini menjamin seluruh labirin terjelajahi.

Itulah **Depth-First Search** (DFS, penelusuran mendalam). Jika BFS (Modul 17) menelusuri secara **melebar** lapis demi lapis, DFS menelusuri secara **mendalam** dari suatu simpul sampai tidak ada lagi simpul yang dapat ditelusuri, lalu melakukan **backtrack**. Selama penelusuran, DFS mencatat dua **timestamp** untuk setiap simpul: kapan simpul itu pertama kali ditemukan dan kapan simpul itu selesai.

Timestamp dan klasifikasi busur yang dihasilkan DFS menjadi dasar banyak algoritma graf lain, misalnya pendeteksian cycle dan pengurutan topologis. Modul ini membahas pseudocode DFS dari bahan kuliah, menelusurinya pada graf berarah enam simpul, dan menafsirkan hasilnya.

PEMBAHASAN INTI

2. Cara Kerja DFS

- DFS menelusuri dari suatu simpul secara mendalam sampai tidak ditemukan lagi simpul yang bisa ditelusuri. Lalu ia **backtrack**: mundur sampai menemukan simpul yang masih memiliki tetangga yang belum ditelusuri (masih ada percabangan sesuai adjacency-list).
- Proses berlanjut sampai semua simpul yang dapat dicapai dari simpul awal ditemukan. Jika masih ada simpul yang tersisa, DFS memilih salah satunya sebagai simpul awal baru, dan begitu seterusnya sampai semua simpul ditemukan.
- BFS dari satu simpul menghasilkan satu **BFS-tree**, sedangkan DFS dapat menghasilkan lebih dari satu tree. Kumpulan tree hasil DFS disebut **DFS-forest**.

2.1 Warna dan timestamp

Atribut	Arti
WHITE	Simpul belum ditemukan (keadaan awal semua simpul).
GRAY	Simpul sudah ditemukan, tetapi adjacency-list-nya belum selesai ditelusuri.
BLACK	Semua simpul dalam adjacency-list-nya sudah ditelusuri; simpul selesai.
$u.d$ (*discovery time*)	Timestamp saat u pertama kali ditemukan; saat itu u menjadi GRAY.
$u.f$ (*finishing time*)	Timestamp saat semua tetangga u selesai ditelusuri; saat itu u menjadi BLACK.
$u.\pi$	Predecessor: simpul yang menemukan u.

Tabel 1. Warna simpul dan timestamp pada DFS.

Timestamp berasal dari variabel global `time` yang bertambah 1 setiap kali ada simpul yang ditemukan atau selesai. Karena setiap simpul ditemukan sekali dan selesai sekali, timestamp bernilai 1 sampai $2|V|$, dan selalu berlaku $u.d < u.f$.

2.2 Pseudocode

DFS(G)

```
for each vertex u ∈ G.V
    u.color = WHITE
    u.π = NIL
time = 0
for each vertex u ∈ G.V
    if u.color == WHITE
        DFS-VISIT(G, u)
```

DFS-VISIT(G, u)

```
time = time + 1
u.d = time
u.color = GRAY
for each v ∈ G.Adj[u]
    if v.color == WHITE
        v.π = u
        DFS-VISIT(G, v)
u.color = BLACK
time = time + 1
u.f = time
```

Keterangan DFS(G). Langkah 1–3: inisialisasi, semua simpul WHITE dan predecessor NIL. Langkah 4: `time = 0`. Langkah 5–7: periksa setiap simpul $u \in G.V$; jika ditemukan simpul WHITE, panggil `DFS-VISIT(G, u)`. Setiap simpul yang dipanggil pada langkah 7 menjadi **akar** dari satu tree di DFS-forest.

Keterangan DFS-VISIT(G, u). Saat dipanggil, u berwarna WHITE. Langkah 1–3: `time` bertambah, $u.d$ dicatat, u menjadi GRAY. Langkah 4–7: telusuri setiap v dalam $Adj[u]$; jika v WHITE, catat $v.\pi = u$ dan panggil `DFS-VISIT(G, v)` secara rekursif. Jika v tidak WHITE, lewati (inilah "backtrack" kecil). Langkah 8–10: setelah semua tetangga selesai, u menjadi BLACK, `time` bertambah, dan $u.f$ dicatat. Setelah itu eksekusi kembali ke fungsi pemanggil, yaitu backtrack ke simpul sebelumnya.

INTUISI

DFS memakai **rekursi** (Modul 02). Tumpukan pemanggilan (call stack) berperan sebagai "jejak langkah" di labirin: puncak tumpukan adalah posisi sekarang, dan `return` berarti mundur satu langkah.

PEMBAHASAN INTI

3. Klasifikasi Busur

Setiap kali DFS-VISIT memeriksa busur (u, v) pada langkah 4, warna v saat itu menentukan jenis busur (Cormen dkk., 2009, Bab 22.3). Bahan kuliah menggambarkan busur non-tree sebagai garis putus-putus berlabel B, F, atau C.

Jenis	Kondisi saat (u, v) diperiksa	Arti
Tree edge	v WHITE	Busur yang menemukan v; menjadi busur DFS-forest ($v.\pi = u$).
Back edge (B)	v GRAY	Menuju leluhur u di tree yang sama, termasuk self-loop. Menandakan adanya **cycle**.
Forward edge (F)	v BLACK dan $u.d < v.d$	Busur non-tree menuju keturunan u.
Cross edge (C)	v BLACK dan $u.d > v.d$	Semua busur lainnya: antar-cabang di tree yang sama atau antar-tree yang berbeda.

Tabel 2. Empat jenis busur hasil DFS pada graf berarah.

CONTOH DAN PENERAPAN

4. Penelusuran DFS pada Graf Berarah

Soal dari bahan kuliah: tuliskan graf berarah dan DFS-forest yang dihasilkan DFS pada graf $G = (V, E)$ dengan adjacency-list berikut.

u	u	v	w	x	y	z
$Adj[u]$	v → x	y	y → z	v	x	z

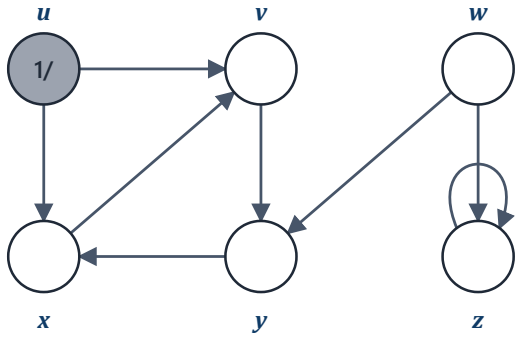
Tabel 3. Adjacency-list graf contoh.

4.1 Jalannya penelusuran

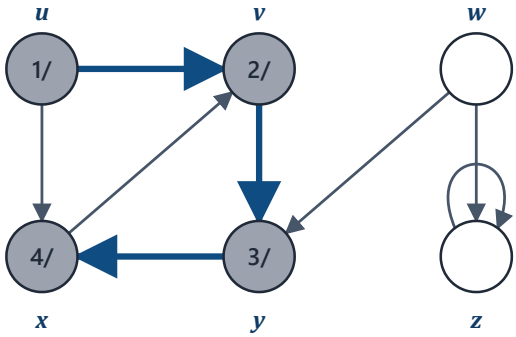
Penelusuran lengkap DFS(G); indentasi menunjukkan kedalaman rekursi

```
DFS langkah 1–4: semua WHITE, π = NIL, time = 0
DFS langkah 5, loop 1: u = u, WHITE → DFS-VISIT(G,u)
  time = 1, u.d = 1, u GRAY
  Adj[u]: v WHITE → (u,v) tree, v.π = u → DFS-VISIT(G,v)
    time = 2, v.d = 2, v GRAY
    Adj[v]: y WHITE → (v,y) tree, y.π = v → DFS-VISIT(G,y)
      time = 3, y.d = 3, y GRAY
      Adj[y]: x WHITE → (y,x) tree, x.π = y → DFS-VISIT(G,x)
        time = 4, x.d = 4, x GRAY
        Adj[x]: v GRAY → (x,v) BACK (B), backtrack
        x BLACK, time = 5, x.f = 5
      y BLACK, time = 6, y.f = 6
    v BLACK, time = 7, v.f = 7
  Adj[u]: x BLACK, u.d = 1 < x.d = 4 → (u,x) FORWARD (F)
  u BLACK, time = 8, u.f = 8                    ← tree pertama selesai (akar u)
DFS loop 2: v BLACK, lewati
DFS loop 3: w WHITE → DFS-VISIT(G,w)
  time = 9, w.d = 9, w GRAY
  Adj[w]: y BLACK, w.d = 9 > y.d = 3 → (w,y) CROSS (C)
  Adj[w]: z WHITE → (w,z) tree, z.π = w → DFS-VISIT(G,z)
    time = 10, z.d = 10, z GRAY
    Adj[z]: z GRAY → (z,z) BACK (B)
    z BLACK, time = 11, z.f = 11
  w BLACK, time = 12, w.f = 12                  ← tree kedua selesai (akar w)
DFS loop 4–6: x, y, z sudah BLACK, lewati. DFS selesai.
```

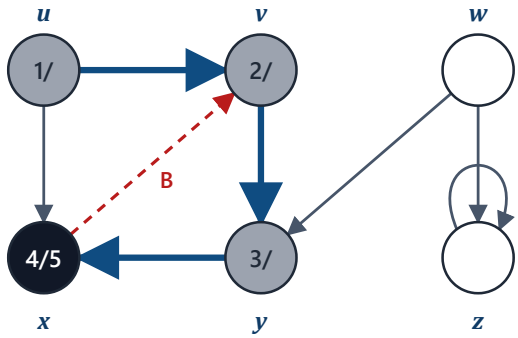
Gambar berikut memperlihatkan keadaan graf pada beberapa saat penting. Label di dalam simpul adalah d/f; busur tebal adalah tree edge; busur merah putus-putus adalah busur non-tree beserta jenisnya.



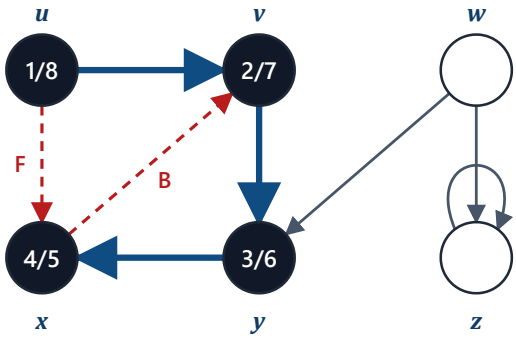
Gambar 1. (a) time = 1: u ditemukan (u.d = 1).



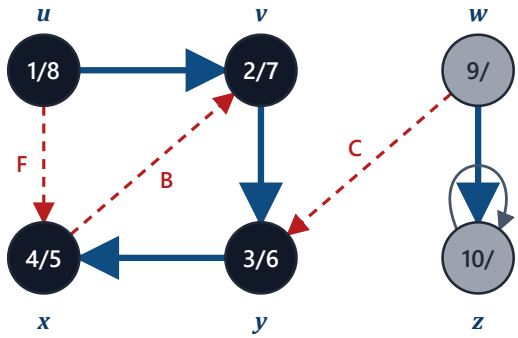
Gambar 2. (b) time = 4: u, v, y, x ditemukan berturut-turut lewat busur tree.



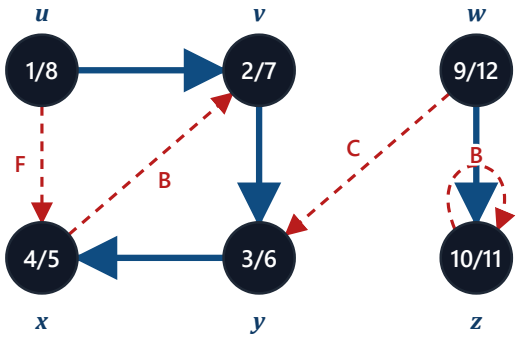
Gambar 3. (c) time = 5: (x, v) ternyata menuju simpul GRAY, jadi busur Back; x selesai.



Gambar 4. (d) time = 8: y, v, u selesai; (u, x) menjadi busur Forward. Tree pertama terbentuk.



Gambar 5. (e) time = 10: w ditemukan; (w, y) busur Cross; z ditemukan.

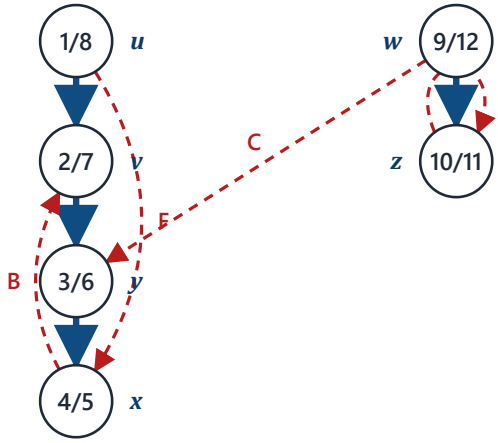


Gambar 6. (f) time = 12: (z, z) busur Back; z dan w selesai. DFS selesai.

4.2 Hasil: DFS-forest

Simpul	u	v	w	x	y	z
d/f	1/8	2/7	9/12	4/5	3/6	10/11
π	NIL	u	NIL	y	v	w

Tabel 4. Timestamp akhir setiap simpul.



Gambar 7. DFS-forest: dua tree dengan akar u dan w. Busur putus-putus adalah busur non-tree (B = back, F = forward, C = cross).

4.3 Struktur tanda kurung

Jika setiap penemuan simpul u ditulis "$(u$" dan setiap penyelesaiannya ditulis "$u)$", urutan kejadian menurut waktu membentuk ekspresi kurung yang tersusun rapi:

$$\underset{1}{(}u\ \underset{2}{(}v\ \underset{3}{(}y\ \underset{4}{(}x\ \underset{5}{x}\underset{6}{)}\ y\underset{7}{)}\ v\underset{8}{)}\ u\underset{9}{)}\ \ \underset{10}{(}w\ \underset{11}{(}z\ \underset{12}{z}\underset{13}{)}\ w\underset{14}{)}$$

Sifat ini disebut **teorema tanda kurung** (Cormen dkk., 2009, Teorema 22.7): untuk dua simpul u dan v, selang $[u.d, u.f]$ dan $[v.d, v.f]$ selalu **bersarang** atau **saling lepas**, tidak pernah tumpang tindih sebagian. Selang bersarang berarti yang satu keturunan yang lain di DFS-forest; contohnya $[4, 5] \subset [2, 7]$ karena x keturunan v. Selang saling lepas berarti keduanya tidak berhubungan leluhur-keturunan; contohnya $[1, 8]$ dan $[9, 12]$.

4.4 Implementasi Python

Python: DFS dengan timestamp dan klasifikasi busur

```python
def dfs(Adj):
    color = {u: "WHITE" for u in Adj}
    d, f, pi, jenis = {}, {}, {u: None for u in Adj}, {}
    time = 0
    def visit(u):
        nonlocal time
        time += 1; d[u] = time; color[u] = "GRAY"
        for v in Adj[u]:
            if color[v] == "WHITE":
                jenis[(u, v)] = "tree"; pi[v] = u; visit(v)
            elif color[v] == "GRAY":
                jenis[(u, v)] = "back"
            else:
                jenis[(u, v)] = "forward" if d[u] < d[v] else "cross"
        color[u] = "BLACK"; time += 1; f[u] = time
    for u in Adj:
        if color[u] == "WHITE":
            visit(u)
    return d, f, pi, jenis

Adj = {"u": ["v", "x"], "v": ["y"], "w": ["y", "z"],
       "x": ["v"], "y": ["x"], "z": ["z"]}
d, f, pi, jenis = dfs(Adj)
print({u: f"{d[u]}/{f[u]}" for u in Adj})
print({f"{a}{b}": j for (a, b), j in jenis.items()})
```

Keluaran program

```
{'u': '1/8', 'v': '2/7', 'w': '9/12', 'x': '4/5', 'y': '3/6', 'z': '10/11'}
{'uv': 'tree', 'vy': 'tree', 'yx': 'tree', 'xv': 'back', 'ux': 'forward', 'wy': 'cross', 'wz':
'tree', 'zz': 'back'}
```

ANALISIS DAN HUBUNGAN KONSEP

5. Running Time dan Perbandingan dengan BFS

5.1 Running time

- Perulangan langkah 1–3 dan 5–7 pada DFS berjalan sekali untuk setiap simpul: $\Theta(V)$, belum termasuk waktu DFS-VISIT.
- DFS-VISIT dipanggil tepat sekali untuk setiap simpul (hanya saat simpul masih WHITE). Di dalamnya, perulangan langkah 4–7 menelusuri $Adj[v]$ sekali. Total panjang semua adjacency-list adalah $\Theta(E)$, sehingga totalnya $\Theta(E)$.

Jadi running time DFS adalah $\Theta(V + E)$.

5.2 DFS dibandingkan dengan BFS

Aspek	BFS (Modul 17)	DFS
Arah penelusuran	Melebar, lapis demi lapis	Mendalam, lalu backtrack
Struktur bantu	Antrean FIFO	Rekursi (call stack, LIFO)
Simpul awal	Satu simpul s	Semua simpul WHITE secara bergiliran
Hasil	BFS-tree (satu tree), jarak $u.d$ dalam jumlah busur	DFS-forest (satu atau lebih tree), timestamp $u.d/u.f$
Arti $u.d$	Jarak dari s	Waktu ditemukan
Running time	$O(V+E)$	$\Theta(V+E)$
Kegunaan khas	Lintasan dengan busur paling sedikit	Deteksi cycle, pengurutan topologis, komponen terhubung kuat

Tabel 5. Perbandingan BFS dan DFS.

Mendeteksi cycle. Graf berarah memiliki cycle jika dan hanya jika DFS menemukan paling sedikit satu back edge (Cormen dkk., 2009, Lemma 22.11). Pada contoh, back edge (x, v) menunjukkan cycle $v \to y \to x \to v$, dan self-loop (z, z) adalah cycle sepanjang 1. Penerapan lain seperti pengurutan topologis dan komponen terhubung kuat dikembangkan dari DFS oleh Tarjan (1972).

KESIMPULAN

6. Kesimpulan

- DFS menelusuri secara mendalam dari setiap simpul WHITE, mundur (backtrack) bila tidak ada lagi tetangga WHITE, dan menghasilkan DFS-forest.
- Setiap simpul memiliki $u.d$ (ditemukan, menjadi GRAY) dan $u.f$ (selesai, menjadi BLACK), bernilai 1 sampai $2|V|$.
- Busur diklasifikasikan berdasarkan warna simpul tujuan saat diperiksa: tree (WHITE), back (GRAY), forward/cross (BLACK).
- Selang $[u.d, u.f]$ bersarang atau saling lepas (teorema tanda kurung); back edge menandakan cycle.
- Pada contoh: u 1/8, v 2/7, y 3/6, x 4/5, w 9/12, z 10/11; DFS-forest berakar u dan w. Running time $\Theta(V+E)$.

Kuis Pemahaman

Kerjakan semua soal terlebih dahulu tanpa melihat kunci jawaban. Kunci jawaban beserta pembahasannya ada di halaman berikutnya.

1 PILIHAN GANDA

Pada DFS, simpul yang berwarna GRAY adalah simpul yang ...

A. Belum ditemukan.
B. Sudah ditemukan, tetapi DFS-VISIT-nya belum selesai (masih berada di call stack).
C. Sudah selesai ditelusuri.
D. Tidak dapat dicapai.

2 PENELUSURAN

Jalankan DFS pada graf berarah Modul 16 dengan adjacency-list $1 : 2, 4$; $2 : 5$; $3 : 6, 5$; $4 : 2$; $5 : 4$; $6 : 6$, dengan urutan simpul 1 sampai 6 pada perulangan DFS. Tuliskan d/f setiap simpul.

3 ANALISIS

Pada soal nomor 2, klasifikasikan busur $(4, 2)$, $(1, 4)$, dan $(3, 5)$.

4 BENAR/SALAH DENGAN ALASAN

"Pada DFS, mungkin saja terjadi $u.d = 2$, $u.f = 7$, $v.d = 5$, $v.f = 9$." Benar atau salah?

5 PERHITUNGAN

Sebuah graf berarah memiliki 7 simpul. Berapa nilai timestamp terbesar yang muncul setelah DFS selesai? Mengapa?

6 ANALISIS KASUS

Pada contoh modul ini, jika urutan perulangan pada DFS(G) diubah menjadi w, u, v, x, y, z, busur (w, y) akan menjadi jenis apa? Berapa banyak tree di DFS-forest?

7 PILIHAN GANDA

Graf berarah tidak memiliki cycle jika dan hanya jika DFS ...

A. Tidak menghasilkan forward edge.
B. Tidak menghasilkan back edge.
C. Hanya menghasilkan satu tree.
D. Tidak menghasilkan cross edge.

8 ANALISIS

Sebutkan dua perbedaan arti atribut $u.d$ pada BFS dan DFS.

Kunci Jawaban dan Pembahasan

Bandingkan jawabanmu, lalu baca pembahasannya untuk memahami alasan di balik setiap jawaban.

Soal 1

Jawaban: B. GRAY berarti simpul sedang "dikunjungi": sudah ditemukan, dan pemanggilan DFS-VISIT-nya masih aktif karena tetangganya belum selesai ditelusuri.

Soal 2

DFS-VISIT(1): $1.d = 1 \rightarrow 2$ ($2.d = 2$) $\rightarrow 5$ ($5.d = 3$) $\rightarrow 4$ ($4.d = 4$); $Adj[4] = 2$ GRAY; $4.f = 5$, $5.f = 6$, $2.f = 7$; kembali ke 1: tetangga 4 BLACK; $1.f = 8$. DFS-VISIT(3): $3.d = 9 \rightarrow 6$ ($6.d = 10$), self-loop; $6.f = 11$; tetangga 5 BLACK; $3.f = 12$. Hasil: **1: 1/8, 2: 2/7, 3: 9/12, 4: 4/5, 5: 3/6, 6: 10/11**.

Soal 3

$(4, 2)$: saat diperiksa, 2 masih GRAY, sehingga **back edge** (cycle $2 \rightarrow 5 \rightarrow 4 \rightarrow 2$). $(1, 4)$: 4 sudah BLACK dan $1.d = 1 < 4.d = 4$, sehingga **forward edge**. $(3, 5)$: 5 BLACK dan $3.d = 9 > 5.d = 3$, sehingga **cross edge**.

Soal 4

Salah. Selang $[2, 7]$ dan $[5, 9]$ tumpang tindih sebagian. Menurut teorema tanda kurung, selang dua simpul selalu bersarang atau saling lepas.

Soal 5

Timestamp terbesar adalah $2 \times 7 = 14$, karena `time` bertambah tepat dua kali untuk setiap simpul: sekali saat ditemukan dan sekali saat selesai.

Soal 6

Dimulai dari w: w 1, lalu y ($w \rightarrow y$ jadi **tree edge**), $y \rightarrow x$, $x \rightarrow v$ (tree), v ke y GRAY (back), lalu z dari w. Kemudian u menjadi akar tree baru, dan (u, v), (u, x) menjadi cross edge. Jadi (w, y) menjadi **tree edge**, dan DFS-forest tetap berisi **2 tree** (akar w dan u), tetapi bentuknya berbeda.

Soal 7

Jawaban: B. Back edge menuju leluhur yang masih GRAY selalu menutup sebuah cycle, dan sebaliknya setiap cycle pasti menghasilkan back edge.

Soal 8

(1) Pada BFS, $u.d$ adalah **jarak** (jumlah busur) dari s; pada DFS, $u.d$ adalah **waktu ditemukan**. (2) Pada BFS nilainya dapat sama untuk banyak simpul (satu lapis), sedangkan pada DFS setiap timestamp unik dan ada pasangannya $u.f$.

Referensi

Sumber utama: Materi 11.2 ALGORITMA GRAF, DFS (bahan kuliah Perancangan dan Analisis Algoritma). Penjelasan tambahan, koreksi, dan pengayaan mengacu pada pustaka berikut.

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). Cambridge, MA: MIT Press. ISBN 978-0-262-03384-8.
2. Tarjan, R. E. (1972). Depth-First Search and Linear Graph Algorithms. *SIAM Journal on Computing*, 1(2), 146–160. https://doi.org/10.1137/0201010
3. Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Upper Saddle River, NJ: Addison-Wesley.